

O'REILLY®

Essential Math for Data Science

Take Control of Your Data with Fundamental
Linear Algebra, Probability, and Statistics



Thomas Nield

Essential Math for Data Science

Master the math needed to excel in data science, machine learning, and statistics. In this book, author Thomas Nield guides you through areas like calculus, probability, linear algebra, and statistics and how they apply to techniques like linear regression, logistic regression, and neural networks. Along the way you'll also gain practical insights into the state of data science and how to use those insights to maximize your career.

Learn how to:

- Use Python code and libraries like SymPy, NumPy, and scikit-learn to explore essential mathematical concepts like calculus, linear algebra, statistics, and machine learning
- Understand techniques like linear regression, logistic regression, and neural networks in plain English, with minimal mathematical notation and jargon
- Perform descriptive statistics and hypothesis testing on a dataset to interpret p-values and statistical significance
- Manipulate vectors and matrices and perform matrix decomposition
- Integrate and build upon incremental knowledge of calculus, probability, statistics, and linear algebra, and apply it to regression models including neural networks
- Navigate practically through a data science career and avoid common pitfalls, assumptions, and biases while tuning your skill set to stand out in the job market

"In the cacophony that is the current data science education landscape, this book stands out as a resource with many clear, practical examples of the fundamentals of what it takes to understand and build with data.

—Vicki Boykis

Senior Machine Learning Engineer
at Tumblr

Thomas Nield is the founder of Nield Consulting Group as well as an instructor at O'Reilly Media and the University of Southern California. He enjoys making technical content relatable and relevant to those unfamiliar or intimidated by it. Thomas regularly teaches classes on data analysis, machine learning, mathematical optimization, and practical artificial intelligence. He's the author of *Getting Started with SQL* (O'Reilly) and *Learning RxJava* (Packt).

DATA

US \$65.99

CAN \$82.99

ISBN: 978-1-098-10293-7



5 6 5 9 9

linkedin.com/company/oreilly-media
youtube.com/oreillymedia

Praise for *Essential Math for Data Science*

In the cacophony that is the current data science education landscape, this book stands out as a resource with many clear, practical examples of the fundamentals of what it takes to understand and build with data. By explaining the basics, this book allows the reader to navigate any data science work with a sturdy mental framework of its building blocks.

—Vicki Boykis, Senior Machine Learning Engineer at Tumblr

Data science is built on linear algebra, probability theory, and calculus. Thomas Nield expertly guides us through all of those topics—and more—to build a solid foundation for understanding the mathematics of data science.

—Mike X Cohen, *sincXpress*

As data scientists, we use sophisticated models and algorithms daily. This book swiftly demystifies the math behind them, so they are easier to grasp and implement.

—Siddharth Yadav, *freelance data scientist*

I wish I had access to this book earlier! Thomas Nield does such an amazing job breaking down complex math topics in a digestible and engaging way. A refreshing approach to both math and data science—seamlessly explaining fundamental math concepts and their immediate applications in machine learning. This book is a must-read for all aspiring data scientists.

—Tatiana Ediger, *freelance data scientist and course developer and instructor*

Essential Math for Data Science

*Take Control of Your Data with Fundamental
Linear Algebra, Probability, and Statistics*

Thomas Nield

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Essential Math for Data Science

by Thomas Nield

Copyright © 2022 Thomas Nield. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: Jessica Haberman

Development Editor: Jill Leonard

Production Editor: Kristen Brown

Copyeditor: Piper Editorial Consulting, LLC

Proofreader: Shannon Turlington

Indexer: Potomac Indexing, LLC

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

June 2022: First Edition

Revision History for the First Edition

2022-05-26: First Release

2024-06-13: Second Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098102937> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Essential Math for Data Science*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author, and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-10293-7

[LSI]

Table of Contents

Preface.....	ix
1. Basic Math and Calculus Review.....	1
Number Theory	2
Order of Operations	3
Variables	5
Functions	6
Summations	11
Exponents	13
Logarithms	16
Euler's Number and Natural Logarithms	18
Euler's Number	18
Natural Logarithms	21
Limits	22
Derivatives	24
Partial Derivatives	28
The Chain Rule	31
Integrals	33
Conclusion	39
Exercises	39
2. Probability.....	41
Understanding Probability	42
Probability Versus Statistics	43
Probability Math	44
Joint Probabilities	44
Union Probabilities	45
Conditional Probability and Bayes' Theorem	47
Joint and Union Conditional Probabilities	49

Binomial Distribution	51
Beta Distribution	53
Conclusion	60
Exercises	61
3. Descriptive and Inferential Statistics.....	63
What Is Data?	63
Descriptive Versus Inferential Statistics	65
Populations, Samples, and Bias	66
Descriptive Statistics	69
Mean and Weighted Mean	70
Median	71
Mode	73
Variance and Standard Deviation	73
The Normal Distribution	78
The Inverse CDF	85
Z-Scores	87
Inferential Statistics	89
The Central Limit Theorem	89
Confidence Intervals	92
Understanding P-Values	95
Hypothesis Testing	96
The T-Distribution: Dealing with Small Samples	104
Big Data Considerations and the Texas Sharpshooter Fallacy	105
Conclusion	107
Exercises	107
4. Linear Algebra.....	109
What Is a Vector?	110
Adding and Combining Vectors	114
Scaling Vectors	116
Span and Linear Dependence	119
Linear Transformations	121
Basis Vectors	121
Matrix Vector Multiplication	124
Matrix Multiplication	129
Determinants	131
Special Types of Matrices	136
Square Matrix	136
Identity Matrix	136
Inverse Matrix	136
Diagonal Matrix	137
Triangular Matrix	137

Sparse Matrix	138
Systems of Equations and Inverse Matrices	138
Eigenvectors and Eigenvalues	142
Conclusion	145
Exercises	146
5. Linear Regression.....	147
A Basic Linear Regression	149
Residuals and Squared Errors	153
Finding the Best Fit Line	157
Closed Form Equation	157
Inverse Matrix Techniques	158
Matrix Decomposition	160
Gradient Descent	161
Overfitting and Variance	167
Stochastic Gradient Descent	169
The Correlation Coefficient	171
Statistical Significance	174
Coefficient of Determination	179
Standard Error of the Estimate	180
Prediction Intervals	181
Train/Test Splits	185
Multiple Linear Regression	191
Conclusion	191
Exercises	192
6. Logistic Regression and Classification.....	193
Understanding Logistic Regression	193
Performing a Logistic Regression	196
Logistic Function	196
Fitting the Logistic Curve	198
Multivariable Logistic Regression	204
Understanding the Log-Odds	208
R-Squared	211
P-Values	216
Train/Test Splits	218
Confusion Matrices	219
Bayes' Theorem and Classification	222
Receiver Operator Characteristics/Area Under Curve	223
Class Imbalance	225
Conclusion	226
Exercises	227

7. Neural Networks.....	229
When to Use Neural Networks and Deep Learning	230
A Simple Neural Network	231
Activation Functions	233
Forward Propagation	239
Backpropagation	245
Calculating the Weight and Bias Derivatives	245
Stochastic Gradient Descent	250
Using scikit-learn	253
Limitations of Neural Networks and Deep Learning	255
Conclusion	258
Exercise	258
8. Career Advice and the Path Forward.....	259
Redefining Data Science	260
A Brief History of Data Science	262
Finding Your Edge	265
SQL Proficiency	265
Programming Proficiency	268
Data Visualization	271
Knowing Your Industry	272
Productive Learning	274
Practitioner Versus Advisor	274
What to Watch Out For in Data Science Jobs	277
Role Definition	277
Organizational Focus and Buy-In	278
Adequate Resources	280
Reasonable Objectives	281
Competing with Existing Systems	282
A Role Is Not What You Expected	284
Does Your Dream Job Not Exist?	285
Where Do I Go Now?	286
Conclusion	287
A. Supplemental Topics.....	289
B. Exercise Answers.....	311
Index.....	325

Preface

In the past 10 years or so, there has been a growing interest in applying math and statistics to our everyday work and lives. Why is that? Does it have to do with the accelerated interest in “data science,” which Harvard Business Review called “**the Sexiest Job of the 21st Century**”? Or is it the promise of machine learning and “artificial intelligence” changing our lives? Is it because news headlines are inundated with studies, polls, and research findings but unsure how to scrutinize such claims? Or is it the promise of “self-driving” cars and robots automating jobs in the near future?

I will make the argument that the disciplines of math and statistics have captured mainstream interest because of the growing availability of data, and we need math, statistics, and machine learning to make sense of it. Yes, we do have scientific tools, machine learning, and other automations that call to us like sirens. We blindly trust these “black boxes,” devices, and softwares; we do not understand them but we use them anyway.

While it is easy to believe computers are smarter than we are (and this idea is frequently marketed), the reality cannot be more the opposite. This disconnect can be precarious on so many levels. Do you really want an algorithm or AI performing criminal sentencing or driving a vehicle, but nobody including the developer can explain why it came to a specific decision? Explainability is the next frontier of statistical computing and AI. This can begin only when we open up the black box and uncover the math.

You may also ask how can a developer not know how their own algorithm works? We will talk about that in the second half of the book when we discuss machine learning techniques and emphasize why we need to understand the math behind the black boxes we build.

To another point, the reason data is being collected on a massive scale is largely due to connected devices and their presence in our everyday lives. We no longer solely use the internet on a desktop or laptop computer. We now take it with us in our

smartphones, cars, and household devices. This has subtly enabled a transition over the past two decades. Data has now evolved from an operational tool to something that is collected and analyzed for less-defined objectives. A smartwatch is constantly collecting data on our heart rate, breathing, walking distance, and other markers. Then it uploads that data to a cloud to be analyzed alongside other users. Our driving habits are being collected by computerized cars and being used by manufacturers to collect data and enable self-driving vehicles. Even “smart toothbrushes” are finding their way into drugstores, which track brushing habits and store that data in a cloud. Whether smart toothbrush data is useful and essential is another discussion!

All of this data collection is permeating every corner of our lives. It can be overwhelming, and a whole book can be written on privacy concerns and ethics. But this availability of data also creates opportunities to leverage math and statistics in new ways and create more exposure outside academic environments. We can learn more about the human experience, improve product design and application, and optimize commercial strategies. If you understand the ideas presented in this book, you will be able to unlock the value held in our data-hoarding infrastructure. This does not imply that data and statistical tools are a silver bullet to solve all the world’s problems, but they have given us new tools that we can use. Sometimes it is just as valuable to recognize certain data projects as rabbit holes and realize efforts are better spent elsewhere.

This growing availability of data has made way for data science and machine learning to become in-demand professions. We define essential math as an exposure to probability, linear algebra, statistics, and machine learning. If you are seeking a career in data science, machine learning, or engineering, these topics are necessary. I will throw in just enough college math, calculus, and statistics necessary to better understand what goes in the black box libraries you will encounter.

With this book, I aim to expose readers to different mathematical, statistical, and machine learning areas that will be applicable to real-world problems. The first four chapters cover foundational math concepts including practical calculus, probability, linear algebra, and statistics. The last three chapters will segue into machine learning. The ultimate purpose of teaching machine learning is to integrate everything we learn and demonstrate practical insights in using machine learning and statistical libraries beyond a black box understanding.

The only tool needed to follow examples is a Windows/Mac/Linux computer and a Python 3 environment of your choice. The primary Python libraries we will need are `numpy`, `scipy`, `sympy`, and `sklearn`. If you are unfamiliar with Python, it is a friendly and easy-to-use programming language with massive learning resources behind it. Here are some I recommend:

Data Science from Scratch, 2nd Edition by Joel Grus (O'Reilly)

The second chapter of this book has the best crash course in Python I have encountered. Even if you have never written code before, Joel does a fantastic job getting you up and running with Python effectively in the shortest time possible. It is also a great book to have on your shelf and to apply your mathematical knowledge!

Python for the Busy Java Developer by Deepak Sarda (Apress)

If you are a software engineer coming from a statically-typed, object-oriented programming background, this is the book to grab. As someone who started programming with Java, I have a deep appreciation for how Deepak shares Python features and relates them to Java developers. If you have done .NET, C++, or other C-like languages you will probably learn Python effectively from this book as well.

This book will not make you an expert or give you PhD knowledge. I do my best to avoid mathematical expressions full of Greek symbols and instead strive to use plain English in its place. But what this book will do is make you more comfortable talking about math and statistics, giving you *essential* knowledge to navigate these areas successfully. I believe the widest path to success is not having deep, specialized knowledge in one topic, but instead having exposure and practical knowledge across several topics. That is the goal of this book, and you will learn just enough to be dangerous and ask those once-elusive critical questions.

So let's get started!

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://github.com/thomasnield/machine-learning-demo-data>.

If you have a technical question or a problem using the code examples, please send email to bookquestions@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Essential Math for Data Science* by Thomas Nield (O'Reilly). Copyright 2022 Thomas Nield, 978-1-098-10293-7.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning



For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-889-8969 (in the United States or Canada)
707-827-7019 (international or local)
707-829-0104 (fax)
support@oreilly.com
<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at <https://oreil.ly/essentialMathDataSci>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly-media>

Watch us on YouTube: <https://youtube.com/oreillymedia>

Acknowledgments

This book was over a year's worth of efforts from many people. First, I want to thank my wife Kimberly for her support while I wrote this book, especially as we raised our son, Wyatt, to his first birthday. Kimberly is an amazing wife and mother, and everything I do now is for my son and our family's better future.

I want to thank my parents for teaching me to struggle past my limits and to never throw in the towel. Given this book's topic, I'm glad they encouraged me to take

calculus seriously in high school and college, and nobody can write a book without regularly exiting their comfort zone.

I want to thank the amazing team of editors and staff at O'Reilly who have continued to open doors since I wrote my first book on SQL in 2015. Jill and Jess have been amazing to work with in getting this book written and published, and I'm grateful that Jess thought of me when this topic came up.

I want to thank my colleagues at University of Southern California in the Aviation Safety and Security program. To have been given the opportunity to pioneer concepts in artificial intelligence system safety has taught me insights few people have, and I look forward to seeing what we continue to accomplish in the years to come. Arch, you continue to amaze me and I worry the world will stop functioning the day you retire.

Lastly, I want to thank my brother Dwight Nield and my friend Jon Ostrower, who are partners in my venture, Yawman Flight. Bootstrapping a startup is hard, and their help has allowed precious bandwidth to write this book. Jon brought me onboard at USC and his tireless accomplishments in the aviation journalism world are nothing short of remarkable (look him up!). It is an honor that they are as passionate as I am about an invention I started in my garage, and I don't think I could bring it to the world without them.

To anybody I have missed, thank you for the big and small things you have done. More often than not, I've been rewarded for being curious and asking questions. I do not take that for granted. As Ted Lasso said, "Be curious, not judgmental."

Basic Math and Calculus Review

We will kick off the first chapter covering what numbers are and how variables and functions work on a Cartesian system. We will then cover exponents and logarithms. After that, we will learn the two basic operations of calculus: derivatives and integrals.

Before we dive into the applied areas of essential math such as probability, linear algebra, statistics, and machine learning, we should probably review a few basic math and calculus concepts. Before you drop this book and run screaming, do not worry! I will present how to calculate derivatives and integrals for a function in a way you were probably not taught in college. We have Python on our side, not a pencil and paper. Even if you are not familiar with derivatives and integrals, you still do not need to worry.

I will make these topics as tight and practical as possible, focusing only on what will help us in later chapters and what falls under the “essential math” umbrella.



This Is Not a Full Math Crash Course!

This is by no means a comprehensive review of high school and college math. If you want that, a great book to check out is *No Bullshit Guide to Math and Physics* by Ivan Savov (pardon my French). The first few chapters contain the best crash course on high school and college math I have ever seen. The book *Mathematics 1001* by Dr. Richard Elwes has some great content as well, and in bite-sized explanations.

Number Theory

What are numbers? I promise to not be too philosophical in this book, but are numbers not a construct we have defined? Why do we have the digits 0 through 9, and not have more digits than that? Why do we have fractions and decimals and not just whole numbers? This area of math where we muse about numbers and why we designed them a certain way is known as number theory.

Number theory goes all the way back to ancient times, when mathematicians studied different number systems, and it explains why we have accepted them the way we do today. Here are different number systems that you may recognize:

Natural numbers

These are the numbers 1, 2, 3, 4, 5...and so on. Only positive numbers are included here, and they are the earliest known system. Natural numbers are so ancient cavemen scratched tally marks on bones and cave walls to keep records.

Whole numbers

Adding to natural numbers, the concept of “0” was later accepted; we call these “whole numbers.” The Babylonians also developed the useful idea for place-holding notation for empty “columns” on numbers greater than 9, such as “10,” “1,000,” or “1,090.” Those zeros indicate no value occupying that column.

Integers

Integers include positive and negative natural numbers as well as 0. We may take them for granted, but ancient mathematicians deeply distrusted the idea of negative numbers. But when you subtract 5 from 3, you get -2 . This is useful especially when it comes to finances where we measure profits and losses. In 628 AD, an Indian mathematician named Brahmagupta showed why negative numbers were necessary for arithmetic to progress with the quadratic formula, and therefore integers became accepted.

Rational numbers

Any number that you can express as a fraction, such as $\frac{2}{3}$, is a rational number. This includes all finite decimals and integers since they can be expressed as fractions, too, such as $\frac{687}{100} = 6.87$ and $\frac{2}{1} = 2$, respectively. They are called *rational* because they are *ratios*. Rational numbers were quickly deemed necessary because time, resources, and other quantities could not always be measured in discrete units. Milk does not always come in gallons. We may have to measure it as parts of a gallon. If I run for 12 minutes, I cannot be forced to measure in whole miles when in actuality I ran $\frac{9}{10}$ of a mile.

Irrational numbers

Irrational numbers cannot be expressed as a fraction. This includes the famous π , square roots of certain numbers like $\sqrt{2}$, and Euler’s number e , which we will

learn about later. These numbers have an infinite number of decimal digits, such as 3.141592653589793238462...

There is an interesting history behind irrational numbers. The Greek mathematician Pythagoras believed all numbers are rational. He believed this so fervently, he made a religion that prayed to the number 10. “Bless us, divine number, thou who generated gods and men!” he and his followers would pray (why “10” was so special, I do not know). There is a legend that one of his followers, Hippasus, proved not all numbers are rational simply by demonstrating the square root of 2. This severely messed with Pythagoras’s belief system, and he responded by drowning Hippasus at sea.

Regardless, we now know not all numbers are rational.

Real numbers

Real numbers include rational as well as irrational numbers. In practicality, when you are doing any data science work you can treat any decimals you work with as real numbers.

Complex and imaginary numbers

You encounter this number type when you take the square root of a negative number. While imaginary and complex numbers have relevance in certain types of problems, we will mostly steer clear of them.

In data science, you will find most (if not all) of your work will be using whole numbers, natural numbers, integers, and real numbers. Imaginary numbers may be encountered in more advanced use cases such as matrix decomposition, which we will touch on in [Chapter 4](#).



Complex and Imaginary Numbers

If you do want to learn about imaginary numbers, there is a great playlist *Imaginary Numbers are Real* on [YouTube](#).

Order of Operations

Hopefully, you are familiar with *order of operations*, which is the order you solve each part of a mathematical expression. As a brief refresher, recall that you evaluate components in parentheses, followed by exponents, then multiplication, division, addition, and subtraction. After that order, operations are then performed left-to-right. You can remember the order of operations by the mnemonic device PEMDAS (Please Excuse My Dear Aunt Sally), which corresponds to the ordering parentheses, exponents, multiplication, division, addition, and subtraction.

Take for example this expression:

$$2 \times \frac{(3 + 2)^2}{5} - 4$$

First we evaluate the parentheses $(3 + 2)$, which equals 5:

$$2 \times \frac{(5)^2}{5} - 4$$

Next we solve the exponent, which we can see is squaring that 5 we just summed. That is 25:

$$2 \times \frac{25}{5} - 4$$

Next up we have multiplication and division. Let's go ahead and multiply the 2 with the $\frac{25}{5}$, yielding $\frac{50}{5}$:

$$\frac{50}{5} - 4$$

Next we will perform the division, dividing 50 by 5, which will yield 10:

$$10 - 4$$

And finally, we perform any addition and subtraction. Of course, $10 - 4$ is going to give us 6:

$$10 - 4 = 6$$

Sure enough, if we were to express this in Python we would print a value of `6.0` as shown in [Example 1-1](#).

Example 1-1. Solving an expression in Python

```
my_value = 2 * (3 + 2)**2 / 5 - 4  
  
print(my_value) # prints 6.0
```

This may be elementary but it is still critical. In code, even if you get the correct result without them, it is a good practice to liberally use parentheses in complex expressions so you establish control of the evaluation order.

Here I group the fractional part of my expression in parentheses, helping to set it apart from the rest of the expression in [Example 1-2](#).

Example 1-2. Making use of parentheses for clarity in Python

```
my_value = 2 * ((3 + 2)**2 / 5) - 4  
print(my_value) # prints 6.0
```

While both examples are technically correct, the latter is more clear to us easily confused humans. If you or someone else makes changes to your code, the parentheses provide an easy reference of operation order as you make changes. This provides a line of defense against code changes to prevent bugs as well.

Variables

If you have done some scripting with Python or another programming language, you have an idea what a variable is. In mathematics, a *variable* is a named placeholder for an unspecified or unknown number.

You may have a variable x representing any real number, and you can multiply that variable without declaring what it is. In [Example 1-3](#) we take a variable input x from a user and multiply it by 3.

Example 1-3. A variable in Python that is then multiplied

```
x = int(input("Please input a number\n"))  
product = 3 * x  
print(product)
```

There are some standard variable names for certain variable types. If these variable names and concepts are unfamiliar, no worries! But some readers might recognize we use theta θ to denote angles and beta β for a parameter in a linear regression. Greek symbols make awkward variable names in Python, so we would likely name these variables theta and beta in Python as shown in [Example 1-4](#).

Example 1-4. Greek variable names in Python

```
beta = 1.75  
theta = 30.0
```

Note also that variable names can be subscripted so that several instances of a variable name can be used. For practical purposes, just treat these as separate variables. If you encounter variables x_1 , x_2 , and x_3 , just treat them as three separate variables as shown in [Example 1-5](#).

Example 1-5. Expressing subscripted variables in Python

```
x1 = 3 # or x_1 = 3  
x2 = 10 # or x_2 = 10  
x3 = 44 # or x_3 = 44
```

Functions

Functions are expressions that define relationships between two or more variables. More specifically, a function takes *input variables* (also called *domain variables* or *independent variables*), plugs them into an expression, and then results in an *output variable* (also called *dependent variable*).

Take this simple linear function:

$$y = 2x + 1$$

For any given x -value, we solve the expression with that x to find y . When $x = 1$, then $y = 3$. When $x = 2$, $y = 5$. When $x = 3$, $y = 7$ and so on, as shown in [Table 1-1](#).

Table 1-1. Different values for $y = 2x + 1$

x	$2x + 1$	y
0	$2(0) + 1$	1
1	$2(1) + 1$	3
2	$2(2) + 1$	5
3	$2(3) + 1$	7

Functions are useful because they model a predictable relationship between variables, such as how many fires y can we expect at x temperature. We will use linear functions to perform linear regressions in [Chapter 5](#).

Another convention you may see for the dependent variable y is to explicitly label it a function of x , such as $f(x)$. So rather than express a function as $y = 2x + 1$, we can also express it as:

$$f(x) = 2x + 1$$

Example 1-6 shows how we can declare a mathematical function and iterate it in Python.

Example 1-6. Declaring a linear function in Python

```
def f(x):  
    return 2 * x + 1  
  
x_values = [0, 1, 2, 3]  
  
for x in x_values:  
    y = f(x)  
    print(y)
```

When dealing with real numbers, a subtle but important feature of functions is they often have an infinite number of x -values and resulting y -values. Ask yourself this: how many x -values can we put through the function $y = 2x + 1$? Rather than just 0, 1, 2, 3...why not 0, 0.5, 1, 1.5, 2, 2.5, 3 as shown in **Table 1-2**?

Table 1-2. Different values for $y = 2x + 1$

x	$2x + 1$	y
0.0	$2(0) + 1$	1
0.5	$2(.5) + 1$	2
1.0	$2(1) + 1$	3
1.5	$2(1.5) + 1$	4
2.0	$2(2) + 1$	5
2.5	$2(2.5) + 1$	6
3.0	$2(3) + 1$	7

Or why not do quarter steps for x ? Or 1/10 of a step? We can make these steps infinitely small, effectively showing $y = 2x + 1$ is a *continuous function*, where for every possible value of x there is a value for y . This segues us nicely to visualize our function as a line as shown in **Figure 1-1**.

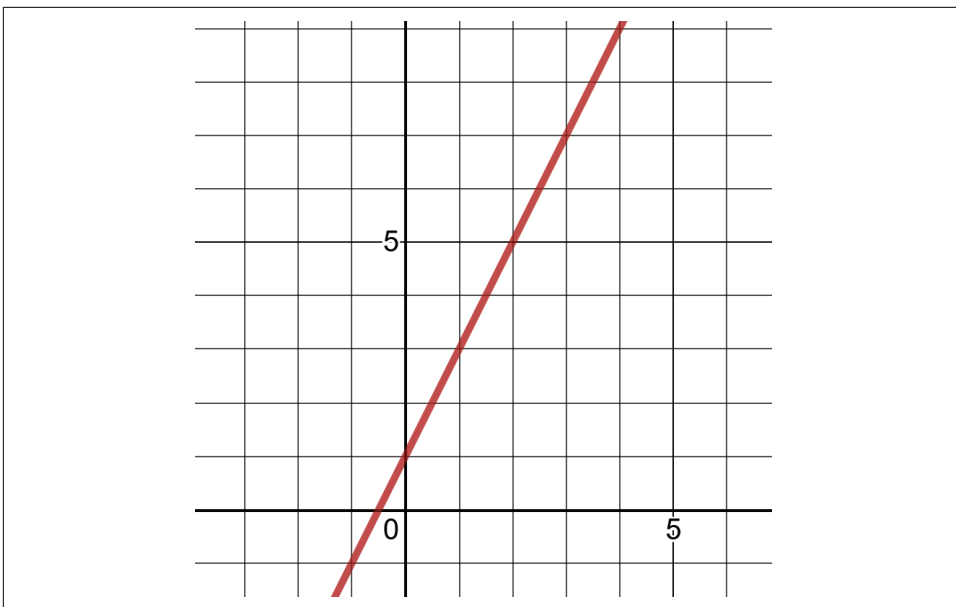


Figure 1-1. Graph for function $y = 2x + 1$

When we plot on a two-dimensional plane with two number lines (one for each variable) it is known as a *Cartesian plane*, *x-y plane*, or *coordinate plane*. We trace a given x-value and then look up the corresponding y-value, and plot the intersections as a line. Notice that due to the nature of real numbers (or decimals, if you prefer), there are an infinite number of x values. This is why when we plot the function $f(x)$ we get a continuous line with no breaks in it. There are an infinite number of points on that line, or any part of that line.

If you want to plot this using Python, there are a number of charting libraries from Plotly to matplotlib. Throughout this book we will use SymPy to do many tasks, and the first we will use is plotting a function. SymPy uses matplotlib so make sure you have that package installed. Otherwise it will print an ugly text-based graph to your console. After that, just declare the x variable to SymPy using `symbols()`, declare your function, and then plot it as shown in [Example 1-7](#) and [Figure 1-2](#).

Example 1-7. Charting a linear function in Python using SymPy

```
from sympy import *

x = symbols('x')
f = 2*x + 1
plot(f)
```

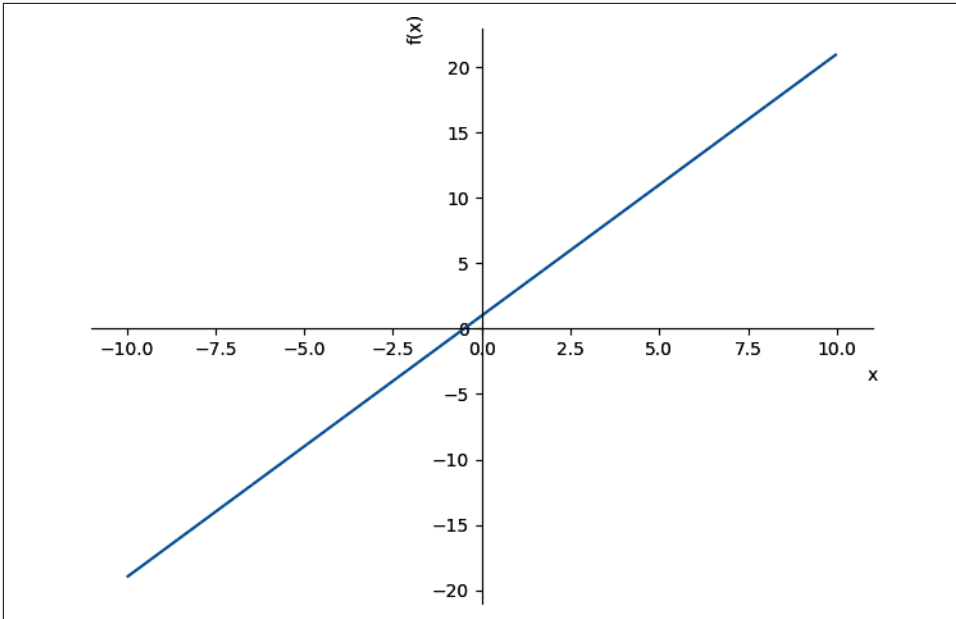


Figure 1-2. Using SymPy to graph a linear function

Example 1-8 and Figure 1-3 are another example showing the function $f(x) = x^2 + 1$.

Example 1-8. Charting a function involving an exponent

```
from sympy import *
x = symbols('x')
f = x**2 + 1
plot(f)
```

Note in Figure 1-3 we do not get a straight line but rather a smooth, symmetrical curve known as a parabola. It is continuous but not linear, as it does not produce values in a straight line. Curvy functions like this are mathematically harder to work with, but we will learn some tricks to make it not so bad.



Curvilinear Functions

When a function is continuous but curvy, rather than linear and straight, we call it a *curvilinear function*.

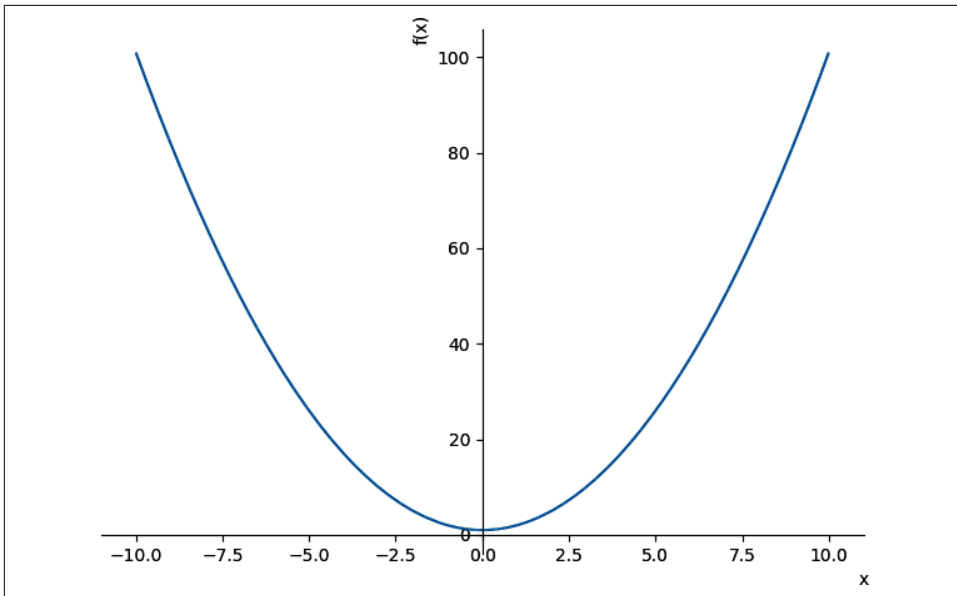


Figure 1-3. Using SymPy to graph a function involving an exponent

Note that functions utilize multiple input variables, not just one. For example, we can have a function with independent variables x and y . Note that y is not dependent like in previous examples.

$$f(x, y) = 2x + 3y$$

Since we have two independent variables (x and y) and one dependent variable (the output of $f(x, y)$), we need to plot this graph on three dimensions to produce a plane of values rather than a line, as shown in [Example 1-9](#) and [Figure 1-4](#).

Example 1-9. Declaring a function with two independent variables in Python

```
from sympy import *
from sympy.plotting import plot3d

x, y = symbols('x y')
f = 2*x + 3*y
plot3d(f)
```

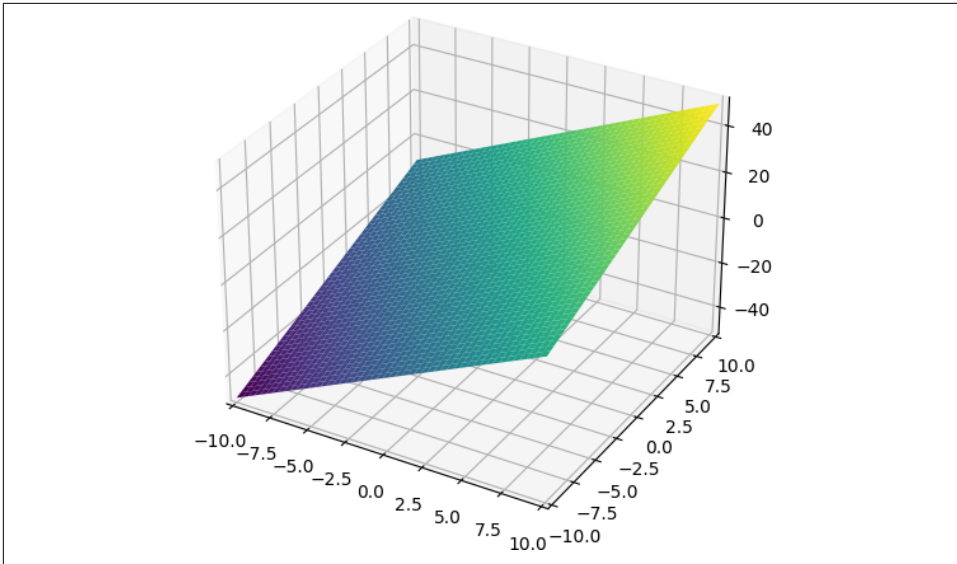


Figure 1-4. Using SymPy to graph a three-dimensional function

No matter how many independent variables you have, your function will typically output only one dependent variable. When you solve for multiple dependent variables, you will likely be using separate functions for each one.

Summations

I promised not to use equations full of Greek symbols in this book. However, there is one that is so common and useful that I would be remiss to not cover it. A *summation* is expressed as a sigma Σ and adds elements together.

For example, if I want to iterate the numbers 1 through 5, multiply each by 2, and sum them, here is how I would express that using a summation. [Example 1-10](#) shows how to execute this in Python.

$$\sum_{i=1}^5 2i = (2)1 + (2)2 + (2)3 + (2)4 + (2)5 = 30$$

Example 1-10. Performing a summation in Python

```
summation = sum(2*i for i in range(1,6))
print(summation)
```

Note that i is a placeholder variable representing each consecutive index value we are iterating in the loop, which we multiply by 2 and then sum all together. When you

are iterating data, you may see variables like x_i indicating an element in a collection at index i .



The range() function

Recall that the `range()` function in Python is end exclusive, meaning if you invoke `range(1,4)` it will iterate the numbers 1, 2, and 3. It excludes the 4 as an upper boundary.

It is also common to see n represent the number of items in a collection, like the number of records in a dataset. Here is one such example where we iterate a collection of numbers of size n , multiply each one by 10, and sum them:

$$\sum_{i=1}^n 10x_i$$

In **Example 1-11** we use Python to execute this expression on a collection of four numbers. Note that in Python (and most programming languages in general) we typically reference items starting at index 0, while in math we start at index 1. Therefore, we shift accordingly in our iteration by starting at 0 in our `range()`.

Example 1-11. Summation of elements in Python

```
x = [1, 4, 6, 2]
n = len(x)

summation = sum(10*x[i] for i in range(0,n))
print(summation)
```

That is the gist of summation. In a nutshell, a summation Σ says, “add a bunch of things together,” and uses an index i and a maximum value n to express each iteration feeding into the sum. We will see these throughout this book.

Summations in SymPy

Feel free to come back to this sidebar later when you learn more about SymPy. SymPy, which we use to graph functions, is actually a symbolic math library; we will talk about what this means later in this chapter. But note for future reference that a summation operation in SymPy is performed using the `Sum()` operator. In the following code, we iterate i from 1 through n , multiply each i , and sum them. But then we use the `subs()` function to specify n as 5, which will then iterate and sum all i elements from 1 through n :

```

from sympy import *

i,n = symbols('i n')

# iterate each element i from 1 to n,
# then multiply and sum
summation = Sum(2*i,(i,1,n))

# specify n as 5,
# iterating the numbers 1 through 5
up_to_5 = summation.subs(n, 5)
print(up_to_5.doit()) # 30

```

Note that summations in SymPy are lazy, meaning they do not automatically calculate or get simplified. So use the `doit()` function to execute the expression.

Exponents

Exponents multiply a number by itself a specified number of times. When you raise 2 to the third power (expressed as 2^3 using 3 as a superscript), that is multiplying three 2s together:

$$2^3 = 2 * 2 * 2 = 8$$

The *base* is the variable or value we are exponentiating, and the *exponent* is the number of times we multiply the base value. For the expression 2^3 , 2 is the base and 3 is the exponent.

Exponents have a few interesting properties. Say we multiplied x^2 and x^3 together. Observe what happens next when I expand the exponents with simple multiplication and then consolidate into a single exponent:

$$x^2 x^3 = (x * x) * (x * x * x) = x^{2+3} = x^5$$

When we multiply exponents together with the same base, we simply add the exponents, which is known as the *product rule*. Let me emphasize that the base of all multiplied exponents must be the same for the product rule to apply.

Let's explore division next. What happens when we divide x^2 by x^5 ?

$$\frac{x^2}{x^5}$$

$$\frac{x^* x}{x^* x^* x^* x^* x}$$

$$\frac{1}{x^* x^* x}$$

$$\frac{1}{x^3} = x^{-3}$$

As you can see, when we divide x^2 by x^5 we can cancel out two x 's in the numerator and denominator, leaving us with $\frac{1}{x^3}$. When a factor exists in both the numerator and denominator, we can cancel out that factor.

What about the x^{-3} , you wonder? This is a good point to introduce negative exponents, which is another way of expressing an exponent operation in the denominator of a fraction. To demonstrate, $\frac{1}{x^3}$ is the same as x^{-3} :

$$\frac{1}{x^3} = x^{-3}$$

Tying back the product rule, we can see it applies to negative exponents, too. To get intuition behind this, let's approach this problem a different way. We can express this division of two exponents by making the "5" exponent of x^5 negative, and then multiplying it with x^2 . When you add a negative number, it is effectively performing subtraction. Therefore, the exponent product rule summing the multiplied exponents still holds up as shown next:

$$\frac{x^2}{x^5} = x^2 \frac{1}{x^5} = x^2 x^{-5} = x^{2 + -5} = x^{-3}$$

Last but not least, can you figure out why any base with an exponent of 0 is 1?

$$x^0 = 1$$

The best way to get this intuition is to reason that any number divided by itself is 1. If you have $\frac{x^3}{x^3}$ it is algebraically obvious that reduces to 1. But that expression also evaluates to x^0 :

$$1 = \frac{x^3}{x^3} = x^3 x^{-3} = x^{3 + -3} = x^0$$

By the transitive property, which states that if $a = b$ and $b = c$, then $a = c$, we know that $x^0 = 1$.

Simplify Expressions with SymPy

If you get uncomfortable with simplifying algebraic expressions, you can use the SymPy library to do the work for you. Here is how to simplify our previous example:

```
from sympy import *  
  
x = symbols('x')  
expr = x**2 / x**5  
print(expr) # x**(-3)
```

Now what about fractional exponents? They are an alternative way to represent roots, such as the square root. As a brief refresher, a $\sqrt{4}$ asks “What number multiplied by itself will give me 4?” which of course is 2. Note here that $4^{1/2}$ is the same as $\sqrt{4}$:

$$4^{1/2} = \sqrt{4} = 2$$

Cubed roots are similar to square roots, but they seek a number multiplied by itself three times to give a result. A cubed root of 8 is expressed as $\sqrt[3]{8}$ and asks “What number multiplied by itself three times gives me 8?” This number would be 2 because $2 * 2 * 2 = 8$. In exponents a cubed root is expressed as a fractional exponent, and $\sqrt[3]{8}$ can be reexpressed as $8^{1/3}$:

$$8^{1/3} = \sqrt[3]{8} = 2$$

To bring it back full circle, what happens when you multiply the cubed root of 8 three times? This will undo the cubed root and yield 8. Alternatively, if we express the cubed root as fractional exponents $8^{1/3}$, it becomes clear we add the exponents together to get an exponent of 1. That also undoes the cubed root:

$$\sqrt[3]{8} * \sqrt[3]{8} * \sqrt[3]{8} = 8^{\frac{1}{3}} \times 8^{\frac{1}{3}} \times 8^{\frac{1}{3}} = 8^{\frac{1}{3} + \frac{1}{3} + \frac{1}{3}} = 8^1 = 8$$

And one last property: an exponent of an exponent will multiply the exponents together. This is known as the *power rule*. So $(8^3)^2$ would simplify to 8^6 :

$$(8^3)^2 = 8^{3 \times 2} = 8^6$$

If you are skeptical why this is, try expanding it and you will see the sum rule makes it clear:

$$(8^3)^2 = 8^3 8^3 = 8^{3+3} = 8^6$$

Lastly, what does it mean when we have a fractional exponent with a numerator other than 1, such as $8^{\frac{2}{3}}$? Well, that is taking the cube root of 8 and then squaring it. Take a look:

$$8^{\frac{2}{3}} = \left(8^{\frac{1}{3}}\right)^2 = 2^2 = 4$$

And yes, irrational numbers can serve as exponents like 8^π , which is 687.2913. This may feel unintuitive, and understandably so! In the interest of time, we will not dive deep into this as it requires some calculus. But essentially, we can calculate irrational exponents by approximating with a rational number. This is effectively what computers do since they can compute to only so many decimal places anyway.

For example π has an infinite number of decimal places. But if we take the first 11 digits, 3.1415926535, we can approximate π as a rational number 31415926535 / 10000000000. Sure enough, this gives us approximately 687.2913, which should approximately match any calculator:

$$8^\pi \approx 8^{\frac{31415926535}{10000000000}} \approx 687.2913$$

Logarithms

A *logarithm* is a math function that finds a power for a specific number and base. It may not sound interesting at first, but it actually has many applications. From measuring earthquakes to managing volume on your stereo, the logarithm is found everywhere. It also finds its way into machine learning and data science a lot. As a matter of fact, logarithms will be a key part of logistic regressions in [Chapter 6](#).

Start your thinking by asking “2 raised to *what power* gives me 8?” One way to express this mathematically is to use an x for the exponent:

$$2^x = 8$$

We intuitively know the answer, $x = 3$, but we need a more elegant way to express this common math operation. This is what the $\log()$ function is for.

$$\log_2 8 = x$$

As you can see in the preceding logarithm expression, we have a base 2 and are finding a power to give us 8. More generally, we can reexpress a variable exponent as a logarithm:

$$a^x = b$$

$$\log_a b = x$$

Algebraically speaking, this is a way of isolating the x , which is important to solve for x . **Example 1-12** shows how we calculate this logarithm in Python.

Example 1-12. Using the log function in Python

```
from math import log

# 2 raised to what power gives me 8?
x = log(8, 2)

print(x) # prints 3.0
```

When you do not supply a base argument to a `log()` function on a platform like Python, it will typically have a default base. In some fields, like earthquake measurements, the default base for the log is 10. But in data science the default base for the log is Euler’s number e . Python uses the latter, and we will talk about e shortly.

Just like exponents, logarithms have several properties when it comes to multiplication, division, exponentiation, and so on. In the interest of time and focus, I will just present this in **Table 1-3**. The key idea to focus on is a logarithm finds an exponent for a given base to result in a certain number.

If you need to dive into logarithmic properties, **Table 1-3** displays exponent and logarithm behaviors side-by-side that you can use for reference.

Table 1-3. Properties for exponents and logarithms

Operator	Exponent property	Logarithm property
Multiplication	$x^m \times x^n = x^{m+n}$	$\log(a \times b) = \log(a) + \log(b)$
Division	$\frac{x^m}{x^n} = x^{m-n}$	$\log\left(\frac{a}{b}\right) = \log(a) - \log(b)$
Exponentiation	$(x^m)^n = x^{mn}$	$\log(a^n) = n \times \log(a)$
Zero Exponent	$x^0 = 1$	$\log(1) = 0$
Inverse	$x^{-1} = \frac{1}{x}$	$\log(x^{-1}) = \log\left(\frac{1}{x}\right) = -\log(x)$

Euler's Number and Natural Logarithms

There is a special number that shows up quite a bit in math called Euler's number e . It is a special number much like Pi π and is approximately 2.71828. e is used a lot because it mathematically simplifies a lot of problems. We will cover e in the context of exponents and logarithms.

Euler's Number

Back in high school, my calculus teacher demonstrated Euler's number in several exponential problems. Finally I asked, "Mr. Nowe, what is e anyway? Where does it come from?" I remember never being fully satisfied with the explanations involving rabbit populations and other natural phenomena. I hope to give a more satisfying explanation here.

Why Euler's Number Is Used So Much

A property of Euler's number is its exponential function is a derivative to itself, which is convenient for exponential and logarithmic functions. We will learn about derivatives later in this chapter. In many applications where the base does not really matter, we pick the one that results in the simplest derivative, and that is Euler's number. That is also why it is the default base in many data science functions.

Here is how I like to discover Euler's number. Let's say you loan \$100 to somebody with 20% interest annually. Typically, interest will be compounded monthly, so the interest each month would be $.20/12 = .01666$. How much will the loan balance be after two years? To keep it simple, let's assume the loan does not require payments (and no payments are made) until the end of those two years.

Putting together the exponent concepts we learned so far (or perhaps pulling out a finance textbook), we can come up with a formula to calculate interest. It consists of a balance A for a starting investment P , interest rate r , time span t (number of years), and periods n (number of months in each year). Here is the formula:

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$

So if we were to compound interest every month, the loan would grow to \$148.69 as calculated here:

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$

$$100 \times \left(1 + \frac{.20}{12}\right)^{12 \times 2} = 148.6914618$$

If you want to do this in Python, try it out with the code in [Example 1-13](#).

Example 1-13. Calculating compound interest in Python

```
from math import exp

p = 100
r = .20
t = 2.0
n = 12

a = p * (1 + (r/n))**(n * t)

print(a) # prints 148.69146179463576
```

But what if we compounded interest daily? What happens then? Change n to 365:

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$

$$100 \times \left(1 + \frac{.20}{365}\right)^{365 \times 2} = 149.1661279$$

Huh! If we compound our interest daily instead of monthly, we would earn 47.4666 cents more at the end of two years. If we got greedy why not compound every hour as shown next? Will that give us even more? There are 8,760 hours in a year, so set n to that value:

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$

$$100 \times \left(1 + \frac{.20}{8760}\right)^{8760 \times 2} = 149.1817886$$

Ah, we squeezed out roughly 2 cents more in interest! But are we experiencing a diminishing return? Let's try to compound every minute! Note that there are 525,600 minutes in a year, so let's set that value to n :

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$

$$100 \times \left(1 + \frac{.20}{525600}\right)^{525600 \times 2} = 149.1824584$$

OK, we are only gaining smaller and smaller fractions of a cent the more frequently we compound. So if I keep making these periods infinitely smaller to the point of compounding continuously, where does this lead?

Let me introduce you to Euler's number e , which is approximately 2.71828. Here is the formula to compound "continuously," meaning we are compounding nonstop:

$$A = P \times e^{rt}$$

Returning to our example, let's calculate the balance of our loan after two years if we compounded continuously:

$$A = P \times e^{rt}$$
$$A = 100 \times e^{.20 \times 2} = 149.1824698$$

This is not too surprising considering compounding every minute got us a balance of 149.1824584. That got us really close to our value of 149.1824698 when compounding continuously.

Typically you use e as an exponent base in Python, Excel, and other platforms using the `exp()` function. You will find that e is so commonly used, it is the default base for both exponent and logarithm functions.

Example 1-14 calculates continuous interest in Python using the `exp()` function.

Example 1-14. Calculating continuous interest in Python

```
from math import exp

p = 100 # principal, starting amount
r = .20 # interest rate, by year
t = 2.0 # time, number of years

a = p * exp(r*t)

print(a) # prints 149.18246976412703
```

So where do we derive this constant e ? Compare the compounding interest formula and the continuous interest formula. They structurally look similar but have some differences:

$$A = P \times \left(1 + \frac{r}{n}\right)^{nt}$$
$$A = P \times e^{rt}$$